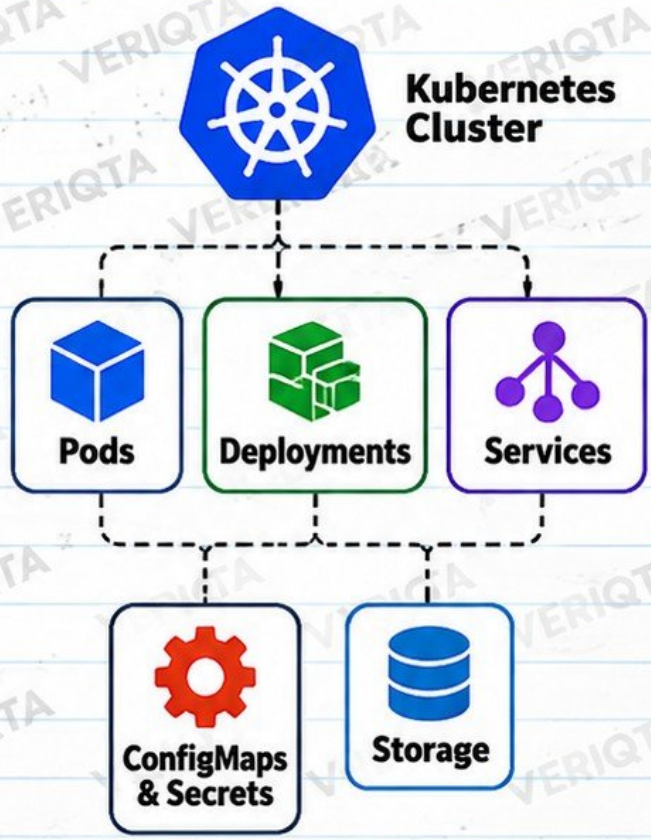
VERIQTA

kubectl

Commands You'll Actually Use

The Beginner's Cheat Sheet

```
$ kubectl get pods
$ kubectl get deployments
$ kubectl get services
$ kubectl describe pod
$ kubectl logs -f
$ kubectl apply -f
$ kubectl rollout status
```



- Essential Commands
- Real Examples
- Troubleshooting Tips
- Production Best Practices
- Quick Reference

get	create	describe	apply	logs	exec	delete
View Resources	Create Resources	Inspect Resources	Deploy with YAML	Read Logs	Run Commands	Remove Resources

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

1. What Is kubectl?

What is kubectl?

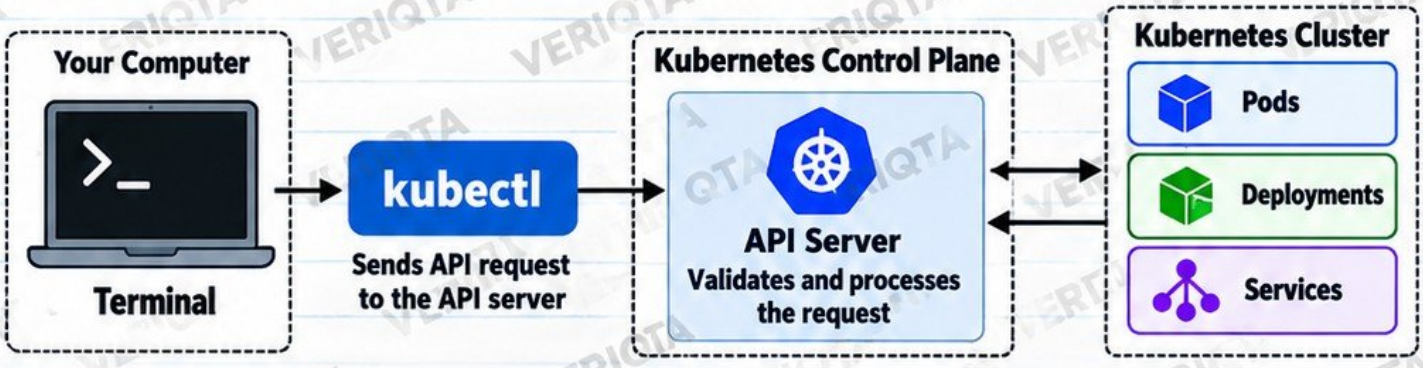
- **kubectl** is the command-line tool used to interact with a **Kubernetes cluster**.
- It allows you to create, view, update, delete and troubleshoot Kubernetes resources such as Pods, Deployments, Services, and more.

Why Engineers Use It

- Manage and inspect applications running in Kubernetes.
- Troubleshoot issues in real time.
- View logs, check resource status and debug problems.
- Make changes to applications and infrastructure.
- It gives direct control over the cluster.

How kubectl Communicates with the Cluster

When you run a **kubectl** command, it sends a request to the **Kubernetes API server**. The API server then communicates with the cluster components to get or make changes.



kubectl Command Structure

```
kubectl [command] [TYPE] [NAME] [flags]
```

- **[command]** Action you want to perform (get, create, delete, etc.).
- **[TYPE]** The resource type (pod, deployment, service, etc.).
- **[NAME]** The name of the resource (optional).
- **[flags]** Additional options to modify the command.

Examples

```
kubectl get pods
kubectl describe deployment nginx
kubectl delete pod my-pod
kubectl apply -f deployment.yaml
kubectl logs my-pod -f
```

Your First kubectl Command

Check the version of your **kubectl** client.

```
$ kubectl version --client

Client Version: v1.28.0
Kustomize Version: v5.0.4
```

What This Does

- Shows the version of your **kubectl** client.
- Confirms that **kubectl** is installed correctly.
- This only shows the client version, not the cluster version.

Next Step

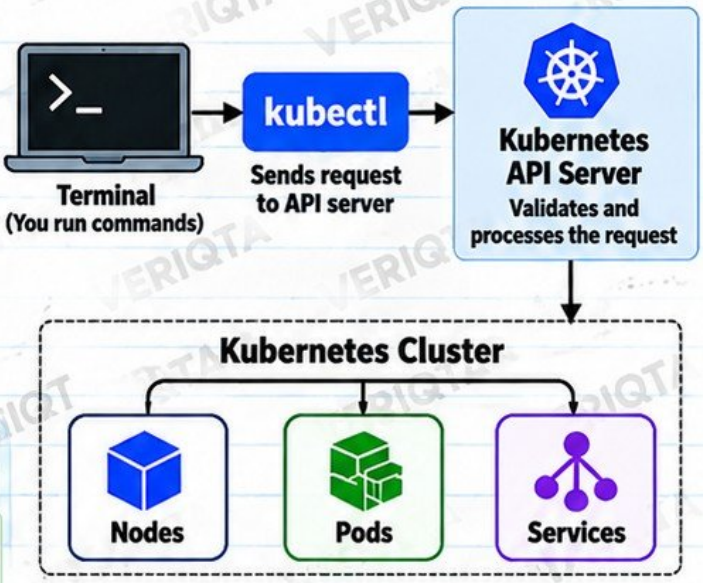
After confirming your client version, connect to your cluster and check the cluster information using: **kubectl cluster-info**

2. Check Your Cluster Connection

Why Check Your Cluster Connection?

- Before you manage or change anything in Kubernetes, you must confirm that your kubectl is connected to the correct cluster.
- This helps you avoid making changes to the wrong cluster or namespace.
- It also ensures that your connection is working and you have access to the resources.

How kubectl Connects to Your Cluster



Essential Commands

1 Check cluster information

```
$ kubectl cluster-info
```

- Shows the addresses of the Kubernetes control plane services, including the API server.
- Confirms that your kubectl can reach the cluster.

2 View your current context

```
$ kubectl config current-context
```

- Shows the name of the cluster, user, and namespace you are currently using.
- Helps you confirm you are working on the right cluster.

3 List all available contexts

```
$ kubectl config get-contexts
```

- Shows all clustres, users, and namespaces configured on your machine.
- The current context is marked with an asterisk (*).

4 List the nodes in your cluster

```
$ kubectl get nodes
```

- Shows the worker and control plane nodes in the cluster.
- You should see nodes in Ready status.

Example Output (Simplified)

```
$ kubectl cluster-info
Kubernetes control plane is running at
https://10.96.0.1:443
CoreDNS is running at
https://10.96.0.10:443
```

```
$ kubectl config current-context
production-cluster
```

```
$ kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	NAMESPACE
*	production	prod-cluster	default
	dev	dev-cluster	dev

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
node-1	Ready	worker	5d	v1.28.0
node-2	Ready	worker	5d	v1.28.0
node-3	Ready	control-plane	5d	v1.28.0

How to Recognize Connection Problems

- Error: connection refused → API server not reachable.
- Error: the server doesn't have a resource type → You may be connected to the wrong API server or an old version.
- Timeout errors → Network issue, VPN, or your cluster is down.
- Unauthorized (401) → Your credentials are invalid or expired.

Why Verifying Context Matters

- You may have multiple clusters (dev, staging, production).
- Running commands on the wrong cluster can cause data loss or service disruption.
- It helps you confirm the correct namespace.
- It ensures you are looking at the right resources before making changes.

3. Find and Inspect Kubernetes Resources

Why Inspect Kubernetes Resources?

- Helps you understand what is running in your cluster.
- You can check the status, configuration and details of each resource.
- It is the first step when troubleshooting an application.

Common Resources You'll Work With

Pods	Deployments	Services	Other Resources
Run your application containers	Manage replicas of Pods	Expose your application to the network	ConfigMaps, Secrets, Jobs and more

Essential Commands

1 List all Pods

```
$ kubectl get pods
```

2 List all Deployments

```
$ kubectl get deployments
```

3 List all Services

```
$ kubectl get services
```

4 List common resources

```
$ kubectl get all
```

5 Get pods with more details

```
$ kubectl get pods -o wide
```

Example Output

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-66d7f	1/1	Running	0	2d
api-5c8f7c	1/1	Running	1	3d

```
$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	3/3	3	3	2d
api	1/1	1	1	3d

```
$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
nginx	ClusterIP	10.96.0.10	80/TCP	2d
api	ClusterIP	10.96.0.20	8080/TCP	3d

```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-66d7f	1/1	Running	0	2d
pod/api-5c8f7c	1/1	Running	1	3d
service/nginx	1	ClusterIP		3d
service/api	1	ClusterIP		3d
deployment.apps/nginx	3/3			2d
deployment.apps/api	1/1			3d

```
$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx-66d7f	1/1	Running	0	2d	10.244.1.5	node-1
api-5c8f7c	1/1	Running	1	3d	10.244.2.8	node-2

Understanding the Output Columns

COLUMN	MEANING
NAME	The nameof the resource (pod, deployment, etc.).
READY	Number of containers ready out of total containers.
STATUS	Current state (Running, Pending, CrashLoopBackOff, etc.).
RESTARTS	How many times the container has restarted.
AGE	How long the resource has been running.
NAMESPACE	The namespace where the resource is running (shown with -A or in wide output).
IP (wide)	The IP address of the Pod.
NODE (wide)	The node where the Pod is running.

Why kubectl get all Does Not List Everything

- It only shows a selected group of common resources such as Pods, Services, Deployments, ReplicaSets, etc.
- Other resource types like ConfigMaps, Secrets, Ingress, Jobs, CronJobs, or Custom Resources are not included.
- You must use specific commands or kubectl api-resources to see all available resource types.

Key Takeaways

- Use the right command to inspect the resource you need.
- Check the status and age to quickly understand the health of your application.
- Use -o wide for additional details like IP and node.
- Remember that get all is helpful but it does not show every Kubernetes resource type.

4. Working with Namespaces

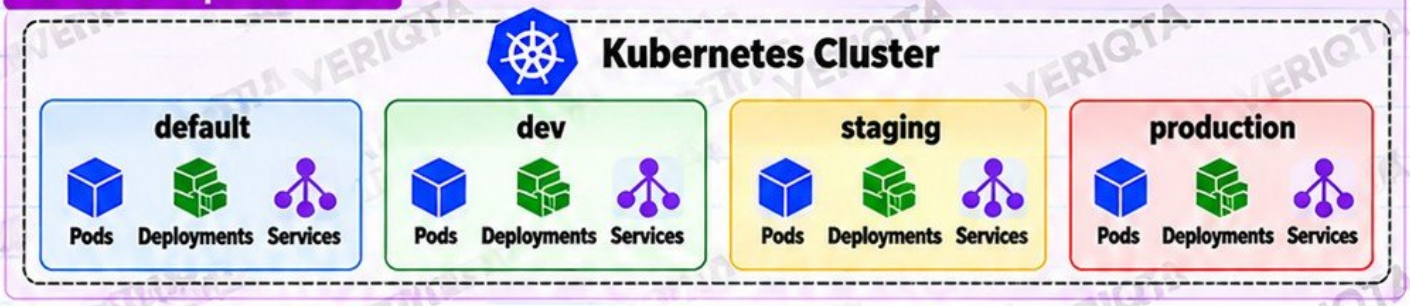
What Are Namespaces?

- A namespace is a logical separation inside a Kubernetes cluster.
- It allows you to organize and isolate resources such as Pods, Deployments, Services, ConfigMaps and more.
- Each namespace has its own set of resources with unique names.
- Namespaces help teams work in the same cluster without affecting each other.

Why Engineers Use Namespaces

- Separate environments (dev, staging, production).
- Isolate teams and projects.
- Prevent naming conflicts.
- Apply different access permissions.
- Keep the cluster organized and easier to manage.
- Reduce the risk of accidental changes in production environments.

How Namespaces Work



Essential Commands

1. List all namespaces

```
$ kubectl get namespaces
```

2. View pods in a specific namespace

```
$ kubectl get pods -n kube-system
```

3. View pods in all namespaces

```
$ kubectl get pods -A
```

4. Create a new namespace

```
$ kubectl create namespace dev
```

5. Set your current namespace

```
$ kubectl config set-context
  --current --namespace=dev
```

Example Output

```
$ kubectl get namespaces
```

NAME	STATUS	AGE
default	Active	5d
kube-system	Active	5d
kube-public	Active	5d
kube-node-lease	Active	5d
dev	Active	1m

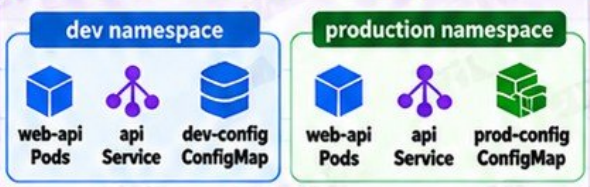
```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-5dd5756b68-abc12	1/1	Running	0	5d
kube-proxy-xyz78	1/1	Running	0	5d

```
$ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	AGE
default	nginx-6d7f	1/1	Running	2d
kube-system	coredns-abc12	1/1	Running	5d
dev	api-7f9d	1/1	Running	1m

Namespace Isolation Example



Important Points

- Resource names only need to be unique within a namespace, not across the entire cluster.
- You can have a Deployment named "api" in both dev and production.
- Use namespaces to organize environments and teams.
- Some system namespaces (e.g. kube-system) run critical cluster components. Be careful when making changes there.

Quick Tips

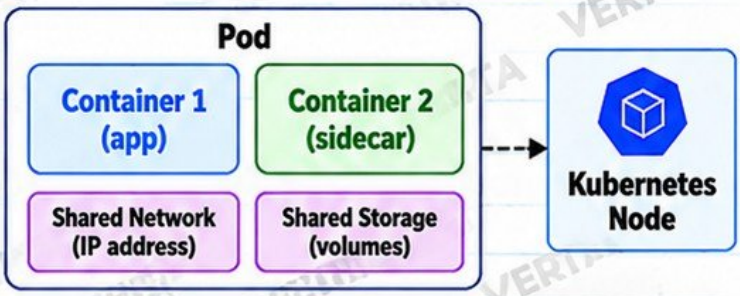
- Use -n <namespace> to target a specific namespace.
- Use -A to view resources in all namespaces.
- Always confirm your current namespace before making changes.
- Keep production workloads in a separate namespace.

5. Inspect Pods Like an Engineer

What is a Pod?

- A Pod is the smallest deployable unit in Kubernetes.
- It can run one or more containers.
- Pods are created and managed by higher-level resources such as Deployments, ReplicaSets or StatefulSets.
- You inspect Pods to check the health of your applications, investigate issues, and understand what is running in the cluster.

Pod Architecture (Simple View)



Essential Commands

1 List all pods

```
$ kubectl get pods
```

2 Inspect pod details

```
$ kubectl describe pod <pod-name>
```

3 View full YAML definition

```
$ kubectl get pod <pod-name> -o yaml
```

Example Output: kubectl get pods

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-66d7f	1/1	Running	0	2d
api-5c8f7c	1/1	Running	1	3d
db-7f9d8	0/1	Pending	0	1m

Example: kubectl describe pod

```
$ kubectl describe pod api-5c8f7c
```

```
Name:         api-5c8f7c
Namespace:    default
Node:         node-1/10.244.1.5
Status:       Running
IP:           10.244.1.20
Containers:
  api
    Image:    my-app:1.0
    State:    Running
    Ready:    True
Events:       <shows recent events here>
```

Pod Phases

PHASE	MEANING
Pending	Pod is scheduled but containers are not running yet.
Running	All containers are running.
Succeeded	All containers completed and exited successfully.
Failed	One or more containers failed.
Unknown	Kubernetes cannot determine the state.

Common Container States

STATE	MEANING
Waiting	Container is starting or waiting for a condition.
Running	Container is running.
Terminated	Container has stopped.

Why a Pod Might Be Pending

- Use enough CPU or memory on nodes.
- No suitable nodes (node selector, taints, or affinity).
- Container image cannot be pulled.
- PersistentVolume claim is not bound.
- Network or CNI issues.

Why a Pod Might Not Be Ready

- Container is crashing or restarting.
- Readiness probe is failing.
- Application inside the container is not responding.
- Missing configuration, secrets, or environment variables.
- Dependency services are not available.

Key Takeaways

- Use kubectl get pods to quickly check the status.
- Use describe to see detailed information and events.
- Use -o yaml to view the full configuration.
- Check the Pod phase, container state, and events when troubleshooting.

6. Read Application Logs

What are Application Logs?

- Logs are output from your application and containers.
- They help you understand what your application is doing.
- Logs are essential for diagnosing failures, debugging issues, and monitoring behavior.

Why Logs Matter?

- Find errors and exceptions.
- Understand why a Pod is crashing.
- Check application health and performance.
- Investigate security or access issues.
- Get insights for better monitoring.

Common kubectl Logs Commands

1 View Logs

```
$ kubectl logs <pod-name>
```

- Shows the logs for the specified Pod.

2 Follow Logs in Real Time

```
$ kubectl logs <pod-name> -f
```

- Streams logs live, like a tail -f.

3 Show Last N Lines

```
$ kubectl logs <pod-name> --tail=50
```

- Shows the last 50 lines of logs.

4 View Previous Logs

```
$ kubectl logs <pod-name> --previous
```

- Shows logs from the previous container instance (after a restart).

5 Specify a Container

```
$ kubectl logs <pod-name> -c <container-name>
```

- Shows logs from a specific container in a Pod.

Example Output

```
$ kubectl logs my-app-pod
2026-10-15T10:15:28Z  INFO   Starting application...
2026-10-15T10:15:29Z  INFO   Connected to database.
2026-10-15T10:15:30Z  ERROR  Failed to process request: database timeout
2026-10-15T10:15:31Z  INFO   Retrying...
```

Real-World Scenarios

- **Application not starting?** Check logs for errors.
- **Pod crashes repeatedly?** Look for exceptions or OOMKilled.
- **Service not responding?** Check application logs for errors or missing dependencies.
- **Everything looks good?** Logs confirm the application is running normally.

Common Issues You'll Find in Logs

Issue	What to Look For
CrashLoopBackOff	Errors, stack traces, or repeated crashes.
ImagePullBackOff	Image not found, wrong tag, or registry issues.
Pending Pod	Scheduling issues, insufficient resources.
Not Ready	Readiness probe failures, connection errors.
Runtime Errors	Exceptions, failed requests, or timeouts.

Key Takeaways

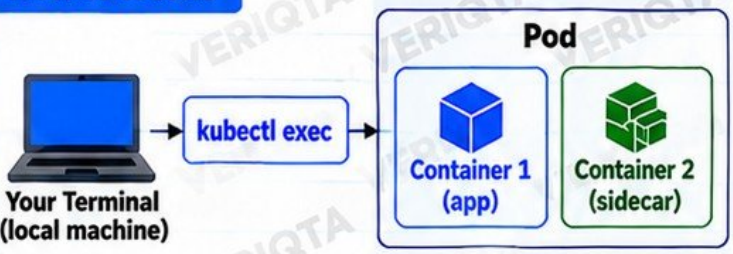
- Use kubectl logs to read application logs.
- Use -f, --tail, and --previous for effective log analysis.
- Check the right container with -c.
- Logs help you find and fix problems faster.

7. Execute Commands Inside Containers

What is kubectl exec?

- **kubectl exec** allows you to run commands **inside a running container** in a Pod.
- It is useful for troubleshooting, inspecting files, checking environment variables, and testing the application.
- You execute the command inside the container, not on your local machine.

How It Works



Common Commands

1 Run a command inside the container

```
$ kubectl exec <pod-name> -- ls /
```

Example Output (listing files in container root)

```
$ kubectl exec nginx-66d7f -- ls /
bin     etc     lib     nginx   sbin    tmp
boot    home    media   opt     srv     usr
dev     lib64   mnt     proc    sys     var
```

2 Open an interactive shell

```
$ kubectl exec -it <pod-name> -- sh
```

Example Output (interactive shell)

```
$ kubectl exec -it nginx-66d7f -- sh
/ # whoami
root
/ #
```

3 Print environment variables

```
$ kubectl exec <pod-name> -- printenv
```

Example Output (environment variables)

```
$ kubectl exec nginx-66d7f -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
HOSTNAME=nginx-66d7f
NGINX_VERSION=1.28.0
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
HOME=/root
```

Local vs Container Commands

Local Command	Container Command
Runs on your machine (laptop or server).	Runs inside the container in the Pod.
Accesses your local files, environment, and tools.	Accesses the container's filesystem, environment, and tools.
Example: ls / (lists files on your machine).	Example: kubectl exec <pod> -- ls / (lists files inside the container).

Why Some Containers Do Not Include a Shell

- Many production containers use minimal images (e.g., distroless, scratch, or Alpine) to:
  - Reduce image size.
  - Improve security (smaller attack surface).
  - Remove unnecessary packages.
- These images may not include a shell like sh or bash.
- If a shell is not available, you can still run specific commands directly, for example:

```
$ kubectl exec <pod-name> -- ls /
$ kubectl exec <pod-name> -- printenv
```

Security Risks

- You get direct access to a running container, which can be risky in production.
- Accidental changes can affect the application.
- Sensitive data (environment variables, credentials, files) may be visible.
- It can be abused if access controls are not properly configured.
- Always follow your organization's security policies and use **kubectl exec** only when necessary.

Key Takeaways

- **kubectl exec** runs a command inside a container, not on your local machine.
- Use it to inspect files, run troubleshooting commands, and check environment variables.
- Minimal container images may not include a shell.
- Be aware of security risks and use **kubectl exec** responsibly.

Common Use Cases

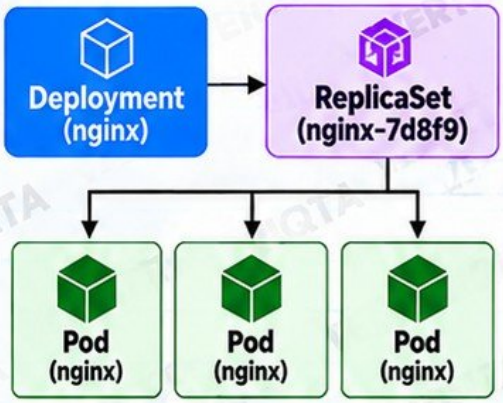
- Check application files and directories.
- Verify environment variables.
- Debug application issues.
- Run database or service specific commands.
- Inspect multiple containers in the same Pod.

8. Create and Manage Deployments

What is a Deployment?

- A **Deployment** manages your application **Pods** and keeps the desired number of replicas running.
- It uses a **ReplicaSet** to create and manage the Pods.
- If a Pod fails, the Deployment automatically replaces it.
- You tell Kubernetes what image to run and how many replicas you want, and it handles the rest.

Deployment → ReplicaSet → Pods



- You create a Deployment.
- Deployment creates and manages a ReplicaSet.
- ReplicaSet creates the required Pods.
- If a Pod fails, ReplicaSet replaces it automatically.

Essential Commands

1 Create a deployment

```
$ kubectl create deployment nginx --image=nginx:stable
```

Example Output

```
deployment.apps/nginx created
```

2 List deployments

```
$ kubectl get deployments
```

Example Output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	1/1	1	1	2m

3 Describe deployment

```
$ kubectl describe deployment nginx
```

Example Output (partial)

```
Name:         nginx
Namespace:    default
Replicas:     1 desired | 1 updated | 1 total | 1 available
Image:        nginx:stable
Selector:     app=nginx
Pods Status:  1 Running
```

4 List replicasets

```
$ kubectl get replicasets
```

Example Output

NAME	DESIRED	CURRENT	READY	AGE
nginx-7d8f9f6d9	1	1	1	2m

Deployment vs Manually Managing Pods

With Deployment (Recommended)	Manually Managing Pods
✓ Automatically keeps the desired number of Pods running.	✗ You must create Pods one by one.
✓ Replaces failed Pods.	✗ If a Pod fails, it will not be replaced automatically.
✓ Supports rolling updates.	✗ No rolling updates or rollbacks.
✓ Supports rollbacks.	✗ Harder to scale.
✓ Easier to scale.	✗ Not suitable for production workloads.
✓ Declarative and production-ready.	

How It Works (Detailed Flow)

You create a Deployment
You specify the container image and desired number of replicas.

↓

Deployment creates a ReplicaSet
The ReplicaSet ensures the correct number of Pods are running.

↓

ReplicaSet creates Pods
Pods run your application containers.

↓

If a Pod fails, it is replaced
The ReplicaSet automatically creates a new Pod to maintain the desired state.

Key Takeaways

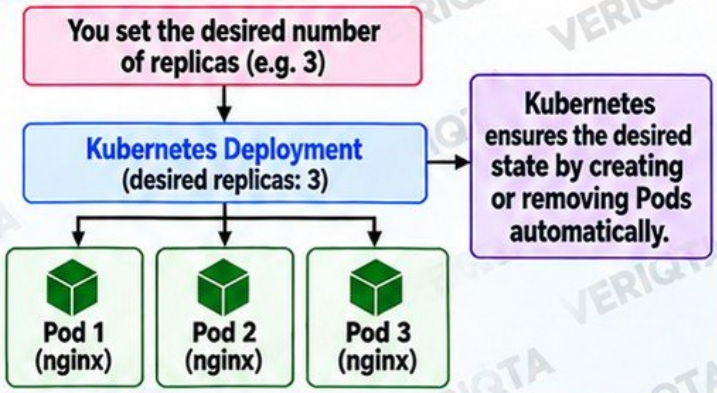
- ✓ Use **kubectl create deployment** to quickly deploy applications.
- ✓ A Deployment manages ReplicaSets and Pods for you.
- ✓ Deployments automatically replace failed Pods.
- ✓ They support rolling updates, rollbacks, and easy scaling.
- ✓ Prefer Deployments over manually creating Pods, especially in production environments.

9. Scale Applications

What Does Scaling Mean?

- Scaling increases or decreases the number of **Pod** replicas running your application.
- It helps your application handle more traffic, improve availability, and recover from failures.
- You tell Kubernetes the desired number of replicas, and it makes sure that number of **Pods** are running.
- Scaling is done on higher-level resources like **Deployments**, not individual Pods.

How Scaling Works



Step-by-Step Commands

1 Scale the deployment to 3 replicas

```
$ kubectl scale deployment/nginx --replicas=3
```

Example Output

```
deployment.apps/nginx scaled
```

2 Check the Pods

```
$ kubectl get pods
```

Example Output

NAME	READY	STATUS	RESTARTS	AGE
nginx-6d4f7d9c7-abc12	1/1	Running	0	10s
nginx-6d4f7d9c7-def34	1/1	Running	0	10s
nginx-6d4f7d9c7-ghi56	1/1	Running	0	10s

3 Check the deployment details

```
$ kubectl get deployment nginx
```

Example Output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	3/3	3	3	1m

4 Watch the rollout status

```
$ kubectl rollout status deployment/nginx
```

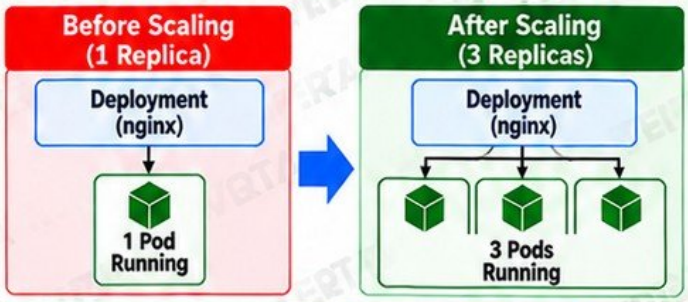
Example Output

```
Waiting for deployment "nginx" rollout to finish:
deployment "nginx" successfully rolled out
```

Replicas and Desired State

- **Replicas**: the number of identical **Pods** running your application.
- **Desired state**: the number of replicas you want.
- Kubernetes continuously checks the actual state and makes sure it matches the desired state.
- If a **Pod** fails, Kubernetes automatically creates a new one to maintain the desired number.

Example: Scaling from 1 to 3 Replicas



Why Increasing Replicas Does Not Always Guarantee Availability

- New Pods might fail to start (image, config, or resource issues).
- Pods might be running but failing health checks.
- The application inside the Pods might not be ready.
- There might be not enough cluster resources (CPU, memory, storage).
- Networking, databases, or external dependencies might be down.
- Readiness probes and startup time can delay availability.

Key Takeaways

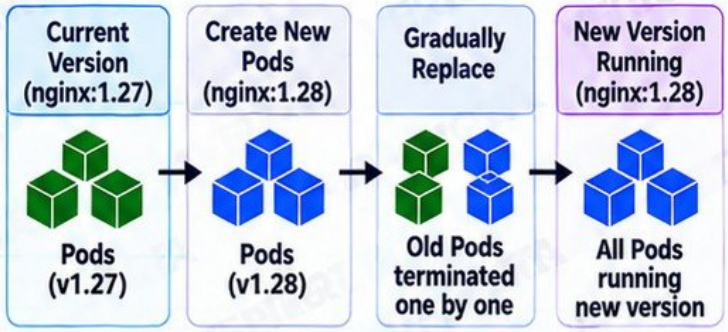
- Use **kubectl scale** to change the number of replicas.
- Kubernetes maintains the desired state.
- Always check Pod status and rollout status after scaling.
- More replicas do not automatically mean the application is healthy.
- Investigate issues if Pods are not ready or the application is unavailable.

10. Update Applications and Roll Back

What Are Rolling Updates?

- A rolling update gradually replaces old Pods with new Pods without downtime.
- Kubernetes creates new Pods with the new version, then slowly removes the old Pods.
- This allows you to update applications safely while keeping them available.

Rolling Update Process



Step-by-Step Commands

1 Update the application image

```
$ kubectl set image deployment/nginx nginx=nginx:1.28
```

Example Output

```
deployment.apps/nginx image updated
```

2 Check the rollout status

```
$ kubectl rollout status deployment/nginx
```

Example Output

```
Waiting for deployment "nginx" rollout to finish:
2 out of 3 new replicas have been updated...
deployment "nginx" successfully rolled out
```

3 View rollout history

```
$ kubectl rollout history deployment/nginx
```

Example Output

```
deployment.apps/nginx
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl set image deployment/nginx nginx=nginx:1.28
```

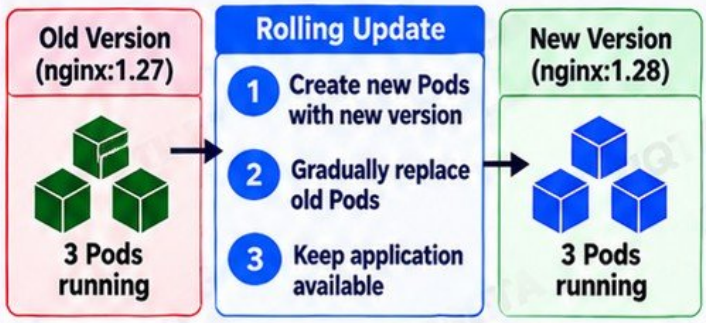
4 Roll back to a previous version

```
$ kubectl rollout undo deployment/nginx
```

Example Output

```
deployment.apps/nginx rolled back
```

Deployment Update Flow



Recognizing Failed Deployments

- New Pods stay in Pending, CrashLoopBackOff, or NotReady.
- Rollout status hangs or shows an error.
- Old Pods may keep running, but new Pods never become healthy.
- Check events and Pod logs to find the cause.

Basic Rollback Limitations

- Rollback only works if a previous ReplicaSet exists.
- It does not fix configuration or database changes.
- If the previous version is also broken, rollback will not help.
- External dependencies (databases, services) may still cause issues after rollback.
- Always test and verify after rolling back.

Key Takeaways

- Use kubectl set image to update a Deployment.
- Rolling updates keep your application available.
- Check rollout status to monitor progress.
- Use rollout history to see past versions.
- Use rollout undo to roll back if needed.
- Recognize failed deployments and investigate the cause.
- Rollback has limitations, so always test and verify.

11. Understand Kubernetes Services

What is a Kubernetes Service?

- A **Service** provides a stable network endpoint to access a set of **Pods**.
- It load balances traffic to healthy **Pods**.
- It hides the underlying **Pods** and their changing IP addresses.
- You can expose your application inside the cluster or to external users using different Service types.

Step-by-Step Commands

1 List services

```
$ kubectl get services
```

Example Output

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	2m
nginx	ClusterIP	10.96.123.45	<none>	80/TCP	1m

2 Describe a service

```
$ kubectl describe service <service-name>
```

Example Output (key details)

```
Name:          nginx
Namespace:     default
Type:          ClusterIP
Cluster IP:    10.96.123.45
Port:          80/TCP
Endpoints:     10.244.1.5:80, 10.244.2.8:80, 10.244.3.6:80
Selector:      app=nginx
```

3 List endpoint slices

```
$ kubectl get endpointslices
```

Example Output

NAME	ADDRESSTYPE	PORTS	AGE
nginx-abc12	IPv4	80	1m
nginx-def34	IPv4	80	1m

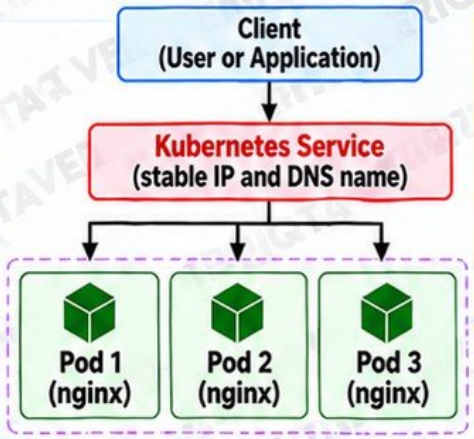
4 Expose a deployment as a service

```
$ kubectl expose deployment nginx --port=80
```

Example Output

```
service/nginx exposed
```

How a Service Works

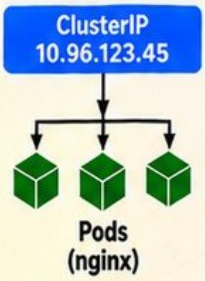


- The Service receives traffic on a stable IP and port.
- It load balances the traffic to one of the healthy Pods.
- If a Pod fails, traffic is sent to the remaining healthy Pods.

Service Types

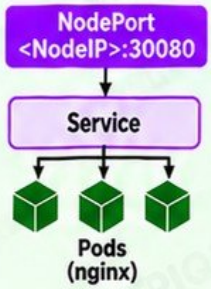
ClusterIP (default)

- Accessible only inside the cluster.
- Gets a virtual IP (ClusterIP).
- Commonly used for internal communication between services.



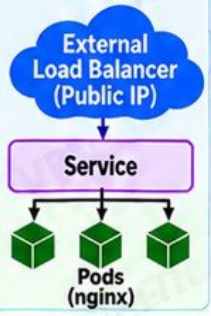
NodePort

- Exposes the service on a static port on every node.
- Accessible from outside the cluster using <NodeIP>:<Port>.
- Useful for testing or simple external access.



LoadBalancer

- Creates an external load balancer (cloud provider).
- Provides a public IP address.
- Distributes traffic to your Pods.
- Suitable for production workloads.



How Services Direct Traffic to Pods

- A **Service** uses selectors (labels) to find matching **Pods**.
- It maintains a list of healthy **Pod** IPs (endpoints).
- It load balances incoming traffic to one of the healthy **Pods**.
- If a **Pod** is removed or fails, the **Service** automatically updates the endpoints and stops sending traffic to that **Pod**.

Key Takeaways

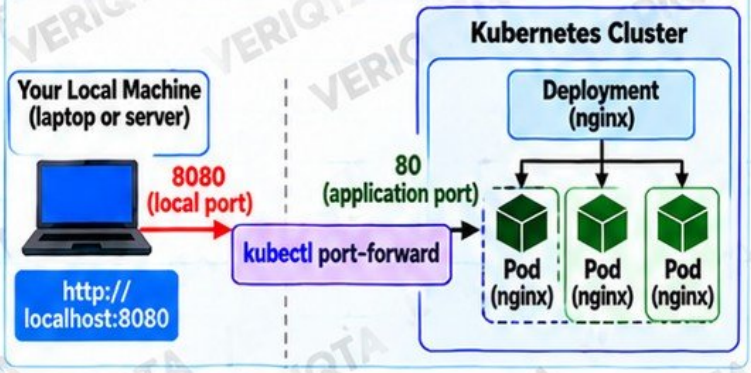
- Use **kubectl get services** to view existing services.
- A **Service** provides a stable endpoint to access your application.
- ClusterIP, NodePort, and LoadBalancer serve different use cases.
- Services direct traffic to healthy **Pods** using selectors and endpoints.
- You can create a **Service** using **kubectl expose**.
- **Services** automatically adapt when **Pods** are added or removed.

12. Access Applications with Port Forwarding

What is Port Forwarding?

- Port forwarding creates a temporary connection from a port on your **local machine** to a **Pod** or **Deployment** inside the Kubernetes cluster.
- It allows you to access your application locally without exposing it publicly.
- It is mainly used for **development, debugging,** and **testing.**

How Port Forwarding Works



Step-by-Step Commands

1 Start port forwarding

```
$ kubectl port-forward deployment/nginx 8080:80
```

Example Output

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
```

2 Test the application locally

```
$ curl http://localhost:8080
```

Example Output (nginx default page)

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

3 Understand local port vs application port

- The **local port (8080)** is the port on your machine that you use to access the application.
- The **application port (80)** is the port your application listens on inside the Pod.
- You can use any available local port, for example: **8080:80, 9090:80,** or **3000:80.**

4 Forward to a specific Pod (optional)

```
$ kubectl port-forward pod/nginx-6d4f7d9c7-abc12 8080:80
```

5 Forward multiple ports (optional)

```
$ kubectl port-forward deployment/nginx 8080:80 8443:443
```

Use Cases

Development
Test your application locally while developing.

Debugging
Troubleshoot issues without exposing the application publicly.

Testing
Quickly verify changes or configuration.

Access internal services
Reach services that are not exposed outside the cluster.

Why Port Forwarding is Useful

- Provides quick local access to your application.
- Helpful for development, debugging, and testing.
- No need to create a Service, Ingress, or expose the application publicly.
- Works with Pods, Deployments, and Services.
- Allows you to access internal services safely.

Why Port Forwarding is NOT a Production Solution

- It is a temporary, local-only connection.
- It is not highly available.
- It is not secure for external users.
- It cannot handle real user traffic or scale.
- It requires an active kubectl session.
- For production, use Services (NodePort/LoadBalancer) and an Ingress with proper security and monitoring.

Key Takeaways

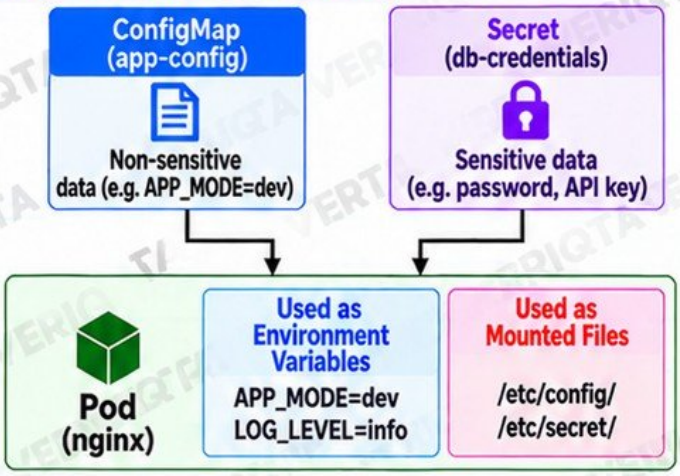
- Use **kubectl port-forward** to access your application locally.
- Test with **curl http://localhost:8080.**
- Understand the difference between local port and application port.
- Use port forwarding for development and debugging.
- Do not use port forwarding as a production ingress solution.

13. Work with ConfigMaps and Secrets

What are ConfigMaps and Secrets?

- **ConfigMaps** store non-sensitive configuration data (e.g. environment variables, config files).
- **Secrets** store sensitive information (e.g. passwords, API keys, tokens).
- Both can be used by **Pods** as environment variables or mounted files.
- They help you separate configuration from application code.

How They Work in a Pod



Step-by-Step Commands

1 List ConfigMaps

```
$ kubectl get configmaps
```

Example Output

```
NAME              DATA   AGE
app-config        2      5m
kube-root-ca.crt  1      10d
```

2 Describe a ConfigMap

```
$ kubectl describe configmap app-config
```

Example Output

```
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_MODE:     dev
LOG_LEVEL:    info
```

3 List Secrets

```
$ kubectl get secrets
```

Example Output

```
NAME              TYPE                                  DATA   AGE
db-credentials    Opaque                                2      5m
default-token-xyz kubernetes.io/service-account-token   3      10d
```

4 Create a ConfigMap

```
$ kubectl create configmap app-config --from-literal=APP_MODE=dev
```

Example Output

```
configmap/app-config created
```

ConfigMap vs Secret

ConfigMap (Non-Sensitive)

- Stores non-sensitive data.
- Used for application configuration.
- Can store key-value pairs or entire config files.
- Safe to view in plain text.
- Example: APP_MODE=dev, LOG_LEVEL=info.

Secret (Sensitive)

- Stores sensitive information.
- Used for passwords, API keys, tokens, certificates.
- Stored in base64 (not real encryption by default).
- Requires access controls (RBAC).
- Should be protected with encryption at rest (e.g. using a cloud provider or KMS).
- Example: DB_PASSWORD, API_TOKEN.

Using ConfigMap and Secret in a Pod (Example)

Use ConfigMap as Environment Variable

```
env:
  - name: APP_MODE
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_MODE
```

Use Secret as Environment Variable

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: db-credentials
        key: password
```

Why Secrets Need Access Controls and Encryption

- Secrets contain sensitive information (passwords, tokens, keys).
- Without access controls, any user or Pod could read them.
- Base64 encoding is NOT encryption (it can be easily decoded).
- Use RBAC to restrict who can access Secrets.
- Use encryption at rest (e.g. cloud provider KMS, etcd encryption).
- Prevent accidental exposure through logs, files, or unsecured access.

Key Takeaways

- Use **ConfigMaps** for non-sensitive configuration data.
- Use **Secrets** for sensitive information.
- Understand the difference between configuration and sensitive data.
- Secrets require access controls and encryption protections.
- Both can be used as environment variables or mounted files in **Pods**.

14. Inspect CPU and Memory Usage

Why Inspect Resource Usage?

- Helps you understand how much CPU and memory your cluster, nodes, and Pods are using.
- Helps identify performance issues, resource pressure, and failing Pods.
- Useful for troubleshooting, capacity planning, and optimization.
- Shows if your applications are using the resources you requested and within the limits you set.

Step-by-Step Commands

1 Check node resource usage

```
$ kubectl top nodes
```

Example Output

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
worker-1	250m	12%	2.1Gi	55%
worker-2	180m	9%	1.4Gi	36%

2 Check pod resource usage

```
$ kubectl top pods
```

Example Output

NAME	CPU(cores)	MEMORY(bytes)
nginx-6d4f7d9c7-abc12	10m	48Mi
api-7c9d8f5b8d-def34	120m	256Mi
worker-5f6b7c8d9e-ghi56	25m	96Mi

3 Describe a node (details)

```
$ kubectl describe node <node-name>
```

Example Output (key details)

```
Name:           worker-1
Capacity:       cpu: 4, memory: 16Gi
Allocatable:    cpu: 3.8, memory: 15Gi
Conditions:     MemoryPressure    False
                DiskPressure      False
                PIDPressure       False
                Ready             True
```

4 Describe a pod (to check resource requests/limits)

```
$ kubectl describe pod <pod-name>
```

Example Output (resources section)

```
Requests:                    Limits:
  cpu:     100m                cpu:     500m
  memory:  128Mi               memory:  512Mi
```

Kubernetes Resource Monitoring Overview

kubectl top (nodes / pods) → Requires Metrics Pipeline (metrics-server)

Nodes (cluster level)



- CPU usage
- Memory usage
- Resource capacity
- Identify node pressure

Pods (application level)



- CPU usage per Pod
- Memory usage per Pod
- Identify high usage Pods
- Find Pods close to limits

CPU and Memory Requests vs Limits

Requests

- Minimum amount of resources guaranteed for the container.
- Used by the scheduler to decide where to place the Pod.
- Helps ensure the Pod has enough resources to run.

Limits

- Maximum amount of resources the container can use.
- Prevents a container from using more than allowed.
- If a container tries to use more than the limit, it can be killed (OOMKilled for memory).

Example	CPU Request	CPU Limit	Memory Request	Memory Limit
	100m	500m	128Mi	512Mi

Understanding OOMKilled

OOMKilled (Out of Memory Killed) happens when a container uses more memory than its limit.

- The kernel terminates the container to protect the node.
- You will see the container state as **Terminated** with reason `OOMKilled` in the pod details.
- Check pod logs and events to confirm the cause.

Why kubectl top Requires a Metrics Pipeline

- **kubectl top** gets data from the **metrics API** (usually provided by **metrics-server**).
- If the metrics server is not installed or not working, you will see an error like:

  `error: Metrics API not available`

- Ensure metrics-server is installed and running in the cluster.

Key Takeaways

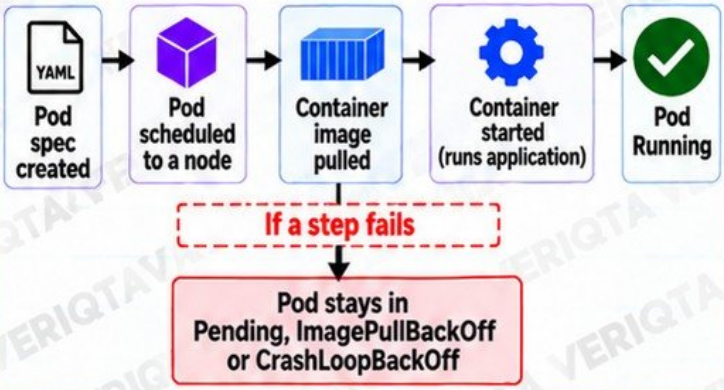
- Use **kubectl top nodes** and **kubectl top pods** to check resource usage.
- Understand the difference between requests and limits.
- Monitor for resource pressure and **OOMKilled** containers.
- Use **kubectl describe** to check node and pod details.
- **kubectl top** requires a working metrics pipeline (metrics-server).
- Proper resource requests and limits help keep your applications stable.

15. Troubleshoot Pods That Will Not Start

Common Reasons Pods Do Not Start

- **Image issues** (wrong image, private registry, pull errors).
- **Configuration errors** (missing ConfigMaps or Secrets).
- **Resource issues** (insufficient CPU or memory).
- **Application errors** (crashes on startup).
- **Scheduling issues** (no available nodes or node constraints).
- **Network or dependency issues** (cannot reach required services).

Pod Startup Flow (What Happens)



Step-by-Step Commands

1 List all Pods

```
$ kubectl get pods
```

Example Output

NAME	READY	STATUS	RESTARTS	AGE
nginx-6d4f7d9c7-abc12	0/1	CrashLoopBackOff	5	2m
api-7c9d8f5b8d-def34	0/1	ImagePullBackOff	0	3m
worker-5f6b7c8d9e-ghi56	0/1	Pending	0	1m

2 Describe a Pod (detailed information)

```
$ kubectl describe pod <pod-name>
```

Example Output (key events)

Type	Reason	Age	Message
Normal	Scheduled	2m	Successfully assigned pod to node
Warning	Failed	1m	Error:ImagePullBackOff
Warning	BackOff	30s	Back-off restarting failed container

3 Check previous container logs

```
$ kubectl logs <pod-name> --previous
```

Example Output

```
Error: failed to start application
Configuration file not found: /app/config.yaml
```

4 Check cluster events (recent)

```
$ kubectl get events --sort-by=.metadata.creationTimestamp
```

Example Output

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m	Warning	FailedScheduling	pod/worker	0/3 nodes are available
1m	Warning	Failed	pod/api	Failed to pull image
30s	Warning	BackOff	pod/nginx	Back-off restarting container

Common Pod Issues and How to Diagnose

CrashLoopBackOff



Container starts, crashes, and restarts repeatedly.

How to Diagnose

- Run `kubectl describe pod <pod-name>`
- Check `kubectl logs <pod-name>`
- Look for application errors (configuration, missing files, bad arguments).
- Check previous logs with `--previous`.

ImagePullBackOff



Kubernetes cannot pull the container image from the registry.

How to Diagnose

- Run `kubectl describe pod <pod-name>`
- Check the image name and tag.
- Verify access to private registry (imagePullSecrets).
- Check network connectivity and registry credentials.
- Look at events for detailed error messages.

Pending



Pod cannot be scheduled to a node.

How to Diagnose

- Run `kubectl describe pod <pod-name>`
- Check events for scheduling errors.
- Look for insufficient resources (CPU/memory).
- Check node selectors, taints, tolerations, and affinities.
- Verify cluster has available nodes.

Use Evidence Before Applying a Fix

- Check pod status, events, and logs.
- Identify the exact reason (image error, crash, resource, etc.).
- Understand what changed before the failure.
- Fix the root cause instead of repeatedly restarting the Pod.
- Verify the fix by checking that the Pod reaches the Running state.

Key Takeaways

- Use `kubectl get pods`, `kubectl describe`, `logs --previous`, and `events` to troubleshoot.
- CrashLoopBackOff = application crash, ImagePullBackOff = image issue, Pending = scheduling/resource issue.
- Always use evidence to find the cause.
- Fix the underlying problem and verify the Pod runs successfully.

16. Troubleshoot Kubernetes Networking

What is Kubernetes Networking?

- Kubernetes networking allows **Pods**, **Services**, and external users to communicate with applications.
- **Services** provide a stable network endpoint and load balance traffic to healthy **Pods**.
- Networking issues can prevent access even when Pods are running.
- Common causes include missing **Service endpoints**, **wrong selectors**, network policies, and configuration errors.

Step-by-Step Commands

1 List all services

```
$ kubectl get services
```

Example Output

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	10d
nginx	ClusterIP	10.96.123.45	<none>	80/TCP	2m

2 List pods with node information

```
$ kubectl get pods -o wide
```

Example Output

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx-6d4f7	1/1	Running	0	2m	10.244.1.10	worker-1
nginx-7c9d8	1/1	Running	0	2m	10.244.2.14	worker-2

3 Describe a service (check selectors and endpoints)

```
$ kubectl describe service <service-name>
```

Example Output

```
Name:          nginx
Namespace:     default
Selector:      app=nginx
Type:          ClusterIP
IP:            10.96.123.45
Port:          80/TCP
Endpoints:     10.244.1.10:80, 10.244.2.14:80
Events:        <none>
```

4 Check endpointslices for the service

```
$ kubectl get endpointslices -l kubernetes.io/service-name=<service-name>
```

Example Output

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
nginx-abc12	IPv4	80	10.244.1.10	2m
nginx-def34	IPv4	80	10.244.2.14	2m

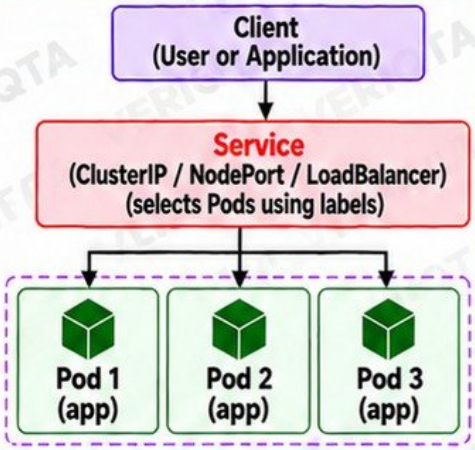
5 Test access using port forwarding

```
kubectl port-forward service/<service-name> 8080:80
```

Example Output

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

How Service Networking Works

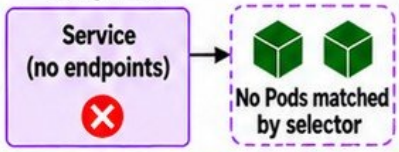


- The Service uses selectors (labels) to find matching Pods.
- It sends traffic to healthy Pods (endpoints).
- If there are no endpoints, the Service cannot forward traffic.

Common Networking Issues and How to Troubleshoot

1. Missing Service Endpoints

- Service shows no endpoints, so it cannot forward traffic to any Pods.

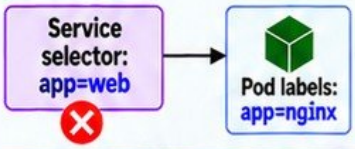


How to Fix

- Check if Pods are running and ready.
- Verify the Service selector matches the Pod labels.
- Check endpointslices.
- Look for label typos or namespace issues.

2. Wrong Selectors

- Service selector does not match any Pod labels, so no endpoints are created.



How to Fix

- Check the selector in `kubectl describe service`.
- Verify Pod labels with `kubectl get pods --show-labels`.
- Update the selector or Pod labels to match.

3. Connection Failures

- Service has endpoints, but applications cannot connect.



How to Fix

- Check that the service port and target port are correct.
- Verify Pods are healthy and listening on the expected port.
- Check network policies, firewall rules, or CNI issues.
- Test using port-forward to isolate the issue.

Key Takeaways

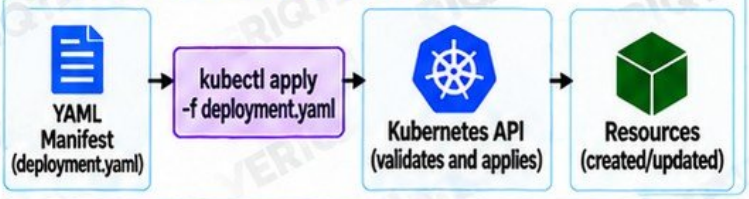
- Use `kubectl get services` to view existing services.
- Use `kubectl get pods -o wide` to check pod IPs and nodes.
- Use `kubectl describe service` to verify selectors, endpoints, and events.
- Use `kubectl get endpointslices` to check the actual endpoints.
- Use `kubectl port-forward` to test access to a service locally.
- Common issues: missing endpoints, wrong selectors, and connection failures.
- Always use evidence (service details, endpoints, events, logs) before applying a fix.

17. Apply and Validate YAML Manifests

What are YAML Manifess?

- YAML manifests define the **desired state** of Kubernetes resources (e.g. **Deployments**, **Services**, **ConfigMaps**).
- You apply the file, and Kubernetes **creates** or **updates** the resources to match the definition.
- This is **declarative resource management** (you define the desired state, not the steps).
- Using YAML files makes your configuration repeatable, version controlled, and easier to manage.

How YAML Manifests Work



Example deployment.yaml (Nginx)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image:  nginx:stable
        ports:
        - containerPort: 80
```

- API version and resource type
- Resource name and labels
- Desired number of replicas
- Selector to match Pods with labels
- Container image and port

Step-by-Step Commands

1 Apply the YAML file

```
$ kubectl apply -f deployment.yaml
```

Example Output

```
deployment.apps/nginx created
# or if already exists
deployment.apps/nginx configured
```

2 Preview changes with diff

```
$ kubectl diff -f deployment.yaml
```

Example Output

```
spec:
  replicas: 2
- replicas: 1
+ replicas: 2
```

3 Validate without applying (server-side)

```
$ kubectl apply --dry-run=server -f deployment.yaml
```

Example Output

```
deployment.apps/nginx (server dry run) configured
```

4 View the live manifest from the cluster

```
$ kubectl get deployment nginx -o yaml
```

Example Output (truncated)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: default
  ...
```

Declarative vs Imperative

Declarative (YAML)	Imperative (Commands)
✓ You define the desired state in a YAML file.	✗ You manually run commands to create or update resources.
✓ Kubernetes automatically creates or updates resources to match the file.	✗ Harder to track and reproduce changes.
✓ Repeatable, version controllable, and consistent.	✗ More prone to configuration drift.

Why Validate Changes Before Applying?

- ✓ Detect errors in the YAML file (invalid fields, wrong API version).
- ✓ See what will change using **kubectl diff**.
- ✓ Test the manifest with **--dry-run=server** to ensure it is valid.
- ✓ Prevent accidental deletion or misconfiguration.
- ✓ Helps you review and understand the impact of the change.

Key Takeaways

- ✓ Use **kubectl apply -f** to create or update resources from a YAML file.
- ✓ Use **kubectl diff -f** to see changes before applying.
- ✓ Use **--dry-run=server** to validate the manifest without making changes.
- ✓ Use **kubectl get -o yaml** to view the current resource definition.
- ✓ YAML manifests provide declarative, repeatable, and version-controlled infrastructure.
- ✓ Always review and validate changes to avoid accidental issues.

18. Delete Resources Safely

Why You Might Delete Resources?

- Remove resources you no longer need.
- Clean up test or temporary environments.
- Recreate resources with new configuration.
- Recover from problems by starting fresh.
- Free up cluster resources (CPU, memory, storage).

Step-by-Step Commands

1 Delete a specific Pod

```
$ kubectl delete pod <pod-name>
```

Example Output

```
pod "nginx-6d4f7d9c7-abc12" deleted
```

2 Delete a Deployment

```
$ kubectl delete deployment nginx
```

Example Output

```
deployment.apps "nginx" deleted
```

3 Delete using a YAML file

```
$ kubectl delete -f deployment.yaml
```

Example Output

```
deployment.apps "nginx" deleted
service "nginx" deleted
```

4 Check Pods in a specific namespace

```
$ kubeectl get pods -n dev
```

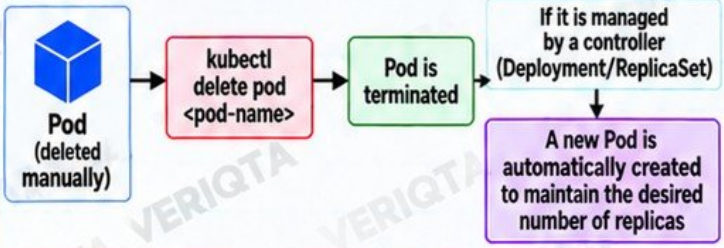
Example Output

NAME	READY	STATUS	RESTARTS	AGE
app-7d9c8f-abc12	1/1	Running	0	2m
app-7d9c8f-def34	1/1	Running	0	2m

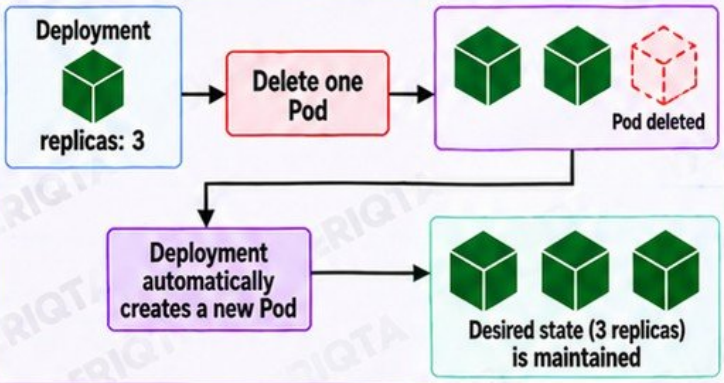
Important Things to Remember

- Always verify what you are deleting.
- Understand the impact on running applications.
- Be aware of controllers that automatically recreate Pods.
- Use namespaces to avoid deleting the wrong resources.
- Consider backups for important data before deletion.

What Happens When You Delete a Pod?



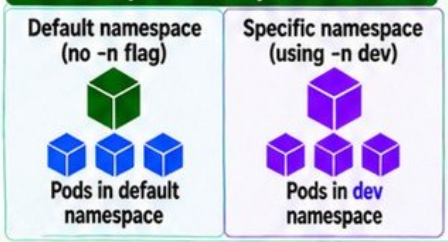
Controller Behavior Example (Deployment)



Common Deletion Risks

- Deleting a Pod managed by a controller will result in a new Pod being created.
- Deleting a Deployment will remove all managed Pods.
- Using kubectl delete -f can delete multiple resources (e.g. Deployment, Service, ConfigMap).
- Applying the wrong file or namespace can delete resources you did not intend to remove.
- Destructive commands can cause application downtime.
- Deleted data (e.g. PersistentVolumes) may not be recoverable depending on your storage setup.

Namespace Scope Matters



Always confirm the namespace before deleting resources.

```
$ kubectl get pods -n dev
```

Key Takeaways

- Use kubectl delete pod <pod-name> to delete a single Pod.
- Use kubectl delete deployment <name> to delete a Deployment.
- Use kubectl delete -f <file> to delete resources defined in a YAML file.
- Controllers will recreate deleted Pods to match the desired state.
- Always check the namespace and understand the impact before deleting.
- Destructive commands require caution to prevent accidental downtime or data loss.

19. Essential kubectl Shortcuts and Productivity

Why Use kubectl Shortcuts?

- Help you work faster and more efficiently.
- Save time when managing and troubleshooting resources.
- Let you discover available resources and their options.
- Provide different output formats for better visibility.
- Built-in help reduces the need to search external documentation.

Step-by-Step Commands with Examples

1 List all Pods (short name)

```
$ kubectl get po
```

Example Output

NAME	READY	STATUS	RESTARTS	AGE
nginx-6d9f7	1/1	Running	0	2m
api-5c7d8	1/1	Running	1	5m

2 List all Deployments (short name)

```
$ kubectl get deploy
```

Example Output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	3/3	3	3	10m
api	2/2	2	2	8m

3 List all Services (short name)

```
$ kubectl get svc
```

Example Output

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	10d
nginx	ClusterIP	10.96.123.45	<none>	80/TCP	5m

4 Get built-in help for a resource

```
$ kubectl explain deployment
```

Example Output

```
KIND:       Deployment
VERSION:    apps/v1
DESCRIPTION:  Deployment enables declarative updates for Pods...
FIELDS:
   spec     <Object>    # Specification of the desired behavior
   status   <Object>    # Most recently observed status
```

5 Discover available resources

```
$ kubectl api-resources
```

Example Output

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
pods	po	v1	true	Pod
services	svc	v1	true	Service
deployments	deploy	apps/v1	true	Deployment

6 Get Pods in JSON format

```
$ kubectl apt pods -o json
```

Example Output

```
{
  "apiVersion": "v1",
  "items": [
    { "metadata": {"name": "nginx-6d9f7", "namespace": "default"},
      "status": {"phase": "Running"}
```

Common kubectl Shortcuts (Quick Reference)

Shortcut	What It Does
kubectl get po	List all Pods (po = short name).
kubectl get deploy	List all Deployments (deploy = short name).
kubectl get svc	List all Services (svc = short name).
kubectl explain <resource>	Show documentation for a resource.
kubectl api-resources	List all available Kubernetes resources.
kubectl get pods -o json	Get Pods output in JSON format.

Output Formats (Use -o)

kubectl supports multiple output formats:

Format	Command Example	Use Case
Table (default)	kubectl get pods	Human-readable table output.
Wide	kubectl get pods -o wide	Shows more columns (e.g. node, IP).
YAML	kubectl get pods -o yaml	Good for reading full resource details.
JSON	kubectl get pods -o json	Useful for scripting and tools.
Custom columns	kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase	

Resource Discovery Examples

List all namespaced resources

```
$ kubectl api-resources
  --namespaced=true
```

Example Output (truncated)

NAME	SHORTNAMES	KIND
pods	po	Pod
services	svc	Service
configmaps	cm	ConfigMap
secrets		Secret
deployments	deploy	Deployment

List all resources in the cluster

```
$ kubectl api-resources
```

Example Output (truncated)

NAME	SHORTNAMES	NAMESPACED
nodes	no	false
pods	po	true
services	svc	true
namespaces	ns	false
deployments	deploy	true

Why These Shortcuts Are Useful

- Quickly check the status of your cluster resources.
- Discover what resources are available and how to use them.
- Use different output formats to get the information you need.
- Built-in help (explain) shows fields, versions, and descriptions.
- Helps with automation, scripting, and debugging.

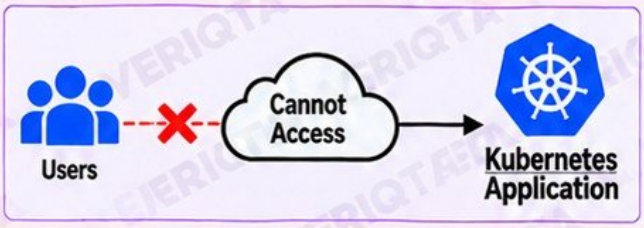
Key Takeaways

- Use short names like po, deploy, and svc to work faster.
- Use kubectl explain to learn about any resource.
- Use kubectl api-resources to discover available resources.
- Use -o yaml, -o json, or -o wide for different output formats.
- These shortcuts improve productivity and help you troubleshoot more effectively.

20. Real-World kubectl Troubleshooting Workflow

Scenario

An application Deployment is running, but users cannot access the application.



Troubleshooting Workflow Overview

1. Verify context and namespace
2. Check Deployment and Pods
3. Inspect events and logs
4. Check Service and EndpointSlices
5. Test connectivity safely
6. Identify the underlying problem
7. Apply a controlled fix
8. Verify application recovery

1 Verify the active cluster context and namespace

Make sure you are working on the correct cluster and namespace.

```
$ kubectl config current-context
$ kubectl get namespaces
$ kubectl config set-context --current --namespace=default
```

2 Check Deployment and Pod status

Verify that the Deployment is running and Pods are healthy.

```
$ kubectl get deployment
$ kubectl get pods
$ kubectl get pods -o wide
```

3 Inspect Pod events and logs

Look for errors, restarts, or crash information.

```
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name>
$ kubectl logs <pod-name> --previous
$ kubectl get events --sort-by=.metadata.creationTimestamp
```

4 Check the Service and EndpointSlices

Confirm the Service is correct and has healthy endpoints.

```
$ kubectl get services
$ kubectl describe service <service-name>
$ kubectl get endpointslices -l kubernetes.io/service-name=<service-name>
```

Final Practical Exercise

Your Task: An nginx application is running, but users cannot access it.

1. Verify the current context and namespace.
2. Check the Deployment and Pod status.
3. Inspect events and logs to find any errors.
4. Check the Service and EndpointSlices.
5. Test connectivity using port-forward.
6. Identify the underlying issue.
7. Apply a fix (e.g. correct selector, update image, or adjust configuration).
8. Verify that the application is accessible again.

5 Test connectivity safely

Use port-forward or a temporary test Pod to check access.

```
$ kubectl port-forward service/<service-name> 8080:80
$ curl http://localhost:8080
```

6 Identify the underlying problem

Use the information from steps 1–5 to find the root cause.

Common Causes

- Pods are not running or crashing
- Service has no endpoints
- Wrong Service selector
- Network policies blocking traffic
- Application error in logs
- Wrong image or configuration
- Resource issues (CPU/Memory)
- DNS or connectivity issues

Use Evidence

- Pod status and logs
- Events and error messages
- Service and EndpointSlices
- Configuration (YAML)
- Network policies
- Resource usage
- Test connectivity results

7 Apply a controlled fix

Fix the specific issue and apply changes carefully.

```
$ kubectl apply -f <fixed-file.yaml>
$ kubectl rollout restart deployment <name>
$ kubectl delete pod <pod-name>   # if a restart is needed
```

8 Verify application recovery

Confirm that the application is working as expected.

```
$ kubectl get pods
$ kubectl get services
$ kubectl get endpointslices -l kubernetes.io/service-name=<service-name>
$ curl http://localhost:8080   # or test via browser
```

Key Takeaways

- Follow a structured workflow to troubleshoot Kubernetes issues.
- Use kubectl commands to gather evidence before making changes.
- Check Deployments, Pods, Services, and EndpointSlices.
- Test connectivity safely using port-forward or a test Pod.
- Identify the real cause using logs, events, and configuration.
- Apply controlled fixes and verify the application recovers.
- Practice the complete workflow to build confidence.